AXI4-Lite Firewall

Access-control and fault-isolation IP core for Intel / Altera Platform Designer

What it is

A synthesisable SystemVerilog core that sits between a bus master and a peripheral you want to protect. Every transaction is checked against a software-programmable table of address ranges before it is forwarded, and every forwarded transaction is watched by a timeout so a wedged peripheral can never hang the master.

There is no stock AXI firewall in the Altera IP catalog, so this is a from-scratch component: RTL, Platform Designer wrapper, self-checking testbench, SystemVerilog assertions, and both a Questa and a licence-free Verilator flow.

Contents

2 - 3 System context: how the core wires into a system
4 - 5 Internal architecture: datapaths, register block, recovery
6 - 7 Datapath state machines
8 - 9 Register map and rule table

Verification status

Questa 2024.1 and Verilator 5.48

Self-checking tests	80 / 80
Assertions	12 / 12
Cover directives	5 / 5
FSM states	8 / 8
FSM transitions	14 / 14
m_axi protocol violations	0

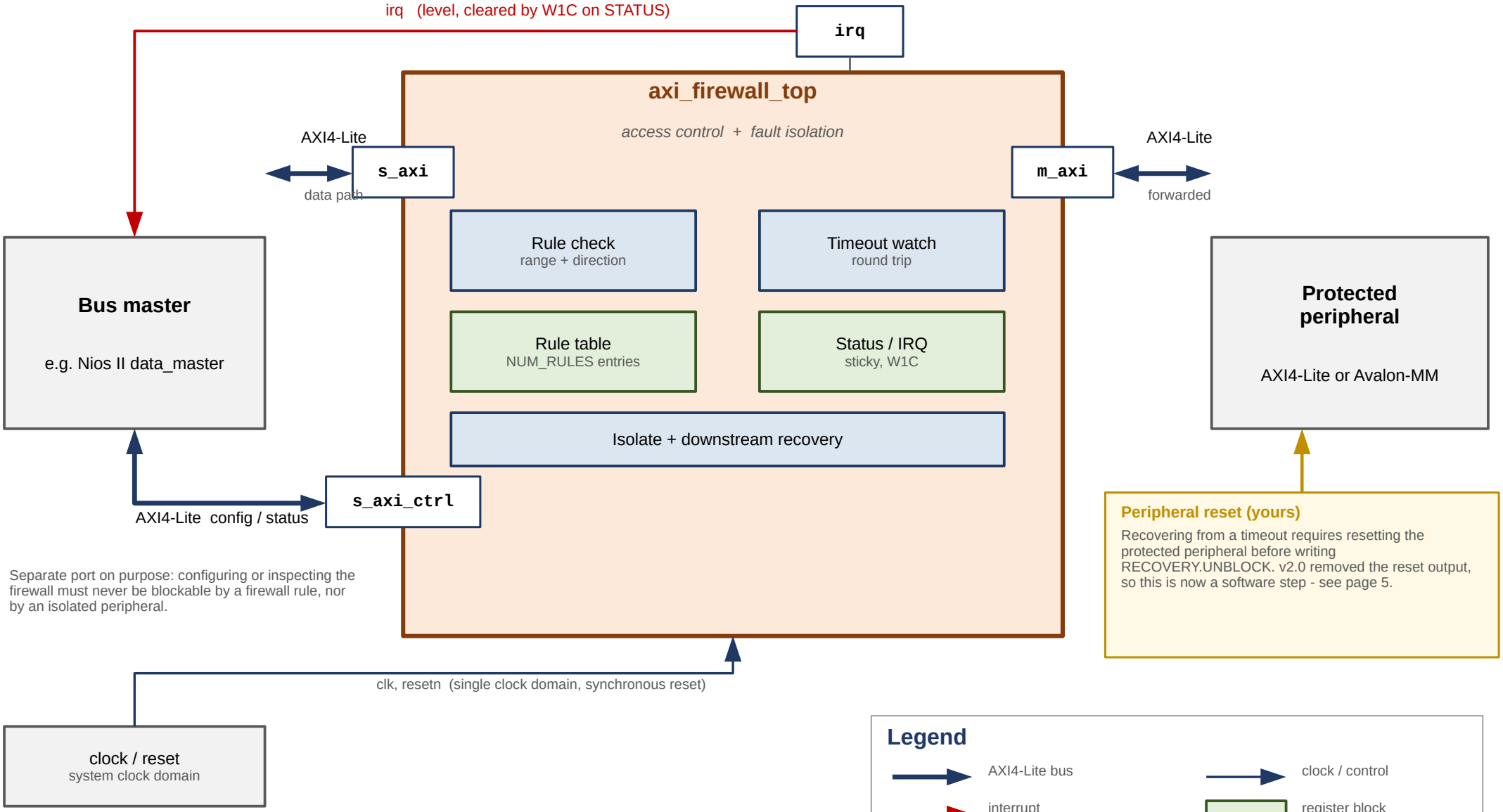
Every assertion has a non-zero non-vacuous pass count.

Not yet verified

Quartus analysis of the hw.tcl component, synthesis results (LE count, Fmax), and behaviour inside a generated Platform Designer interconnect.

1. System context

Where the core sits, and what must be connected to it.



1. System context — description

Purpose

The core is a bump-in-the-wire between one master and one protected peripheral. It enforces an allow-list on every transaction and guarantees the master always gets a response, even when the peripheral stops answering.

Access control

Each transaction's address is compared against a table of ranges, each with independent read and write permission. The lowest-index valid rule containing the address wins; ranges need not be disjoint, so put more specific rules at lower indices. Default-deny applies:

```
no rule matches            →  DECERR
rule matches, wrong direction  →  SLVERR
blocked while ISOLATED         →  SLVERR
```

A denied read returns zeroed RDATA rather than stale bus data, so a rejected read cannot leak the result of an earlier permitted one.

Interface summary

Interface	Type	Connects to
s_axi	slave	the master being policed
m_axi	master	the protected peripheral
s_axi_ctrl	slave	rule table, status, IRQ enable
irq	interrupt	a CPU interrupt input

Fault isolation

Every forwarded transaction is watched by a timeout covering the whole round trip, address issue to response. That catches a peripheral that never raises AWREADY/ARREADY as well as one that accepts and then goes quiet. On expiry the core answers SLVERR upstream immediately, so the master never hangs, and latches an internal broken state that blocks all further forwarding until software acknowledges the fault.

Why the control port is separate

s_axi_ctrl is a physically distinct AXI4-Lite slave. Recovery requires writing STATUS, and if that write had to pass through the firewall's own rule check, or through an isolated peripheral, a fault would be unrecoverable. Both ports may be driven by the same master.

Recovery is a software sequence (v2.0)

ack the fault → poll the busy bits (bounded) → RESET THE PERIPHERAL → write RECOVERY.UNBLOCK

UNBLOCK is what withdraws a stuck VALID. Skipping the reset makes that a protocol violation on a live bus: measured, 1 of 25 timing offsets then mis-attributes a stale response, against 0 of 25 when the sequence is followed.

Known limits

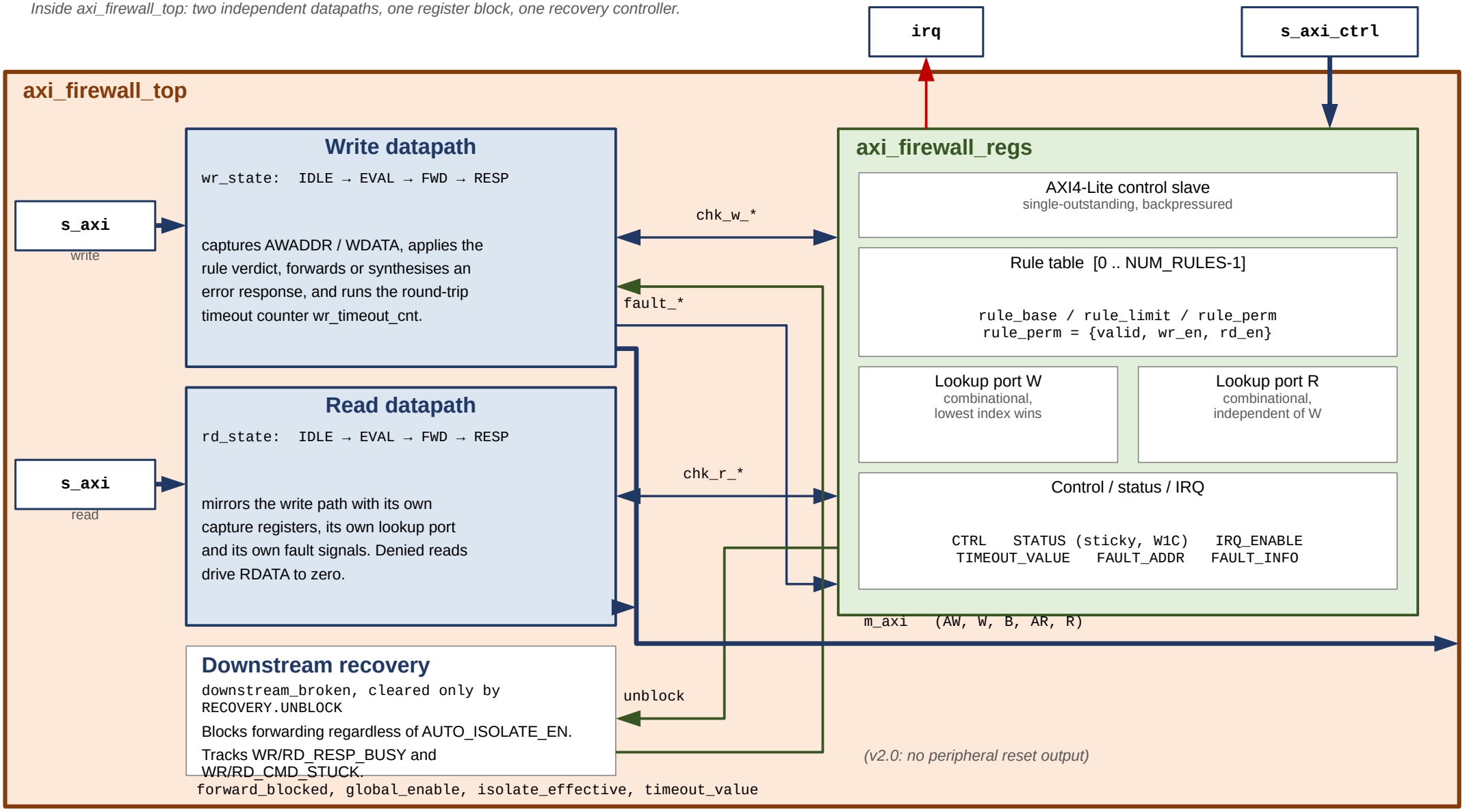
No per-master filtering. AXI4-Lite carries no ID field, so the core sees only the address and the direction, never who asked. That needs full AXI4 or a sideband ID signal.

Single-outstanding, non-pipelined, no bursts. Fine for register-style peripherals a CPU pokes occasionally; a burst master pays the full per-beat cost.

ISOLATE blocks new transactions only. Work already forwarded finishes or times out normally.

2. Internal architecture

Inside axi_firewall_top: two independent datapaths, one register block, one recovery controller.



`chk_*_` is the rule-lookup bundle: `chk_*_addr` out to the register block, `chk_*_match` and `chk_*_allow` back, all combinational within one cycle.

The two datapaths are fully independent: separate FSMs, separate capture registers, separate lookup ports, separate fault pulses. A read and a write can be in flight at the same time. Only FAULT_ADDR / FAULT_INFO are shared, and if both fault in the same cycle the write side wins - a documented, deterministic tie-break.

2. Internal architecture — description

Datapaths

Both follow the same four-state shape. IDLE waits for a request and captures it; EVAL applies the verdict in a single cycle; FWD drives the master side and runs the timeout counter; RESP holds the response until the master accepts it.

EVAL is where policy is decided, in strict priority order: forwarding blocked (SLVERR), global bypass (forward unconditionally), no rule match (DECERR), rule matched but direction denied (SLVERR), otherwise forward.

Register block

Owns the rule table and all software-visible state, and exposes two independent purely combinational lookup ports so both datapaths get an answer in the same cycle without contending. The lookup is a priority chain over NUM_RULES entries - it is the likeliest critical path, and the standard fix if it limits Fmax is to register it with an extra pipeline stage.

Timeout and recovery

On expiry the core reports SLVERR upstream, latches downstream_broken, and deliberately leaves any asserted m_axi_*VALID asserted. AXI requires VALID to hold until READY; withdrawing it can wedge the interconnect between the core and the peripheral, not just the peripheral.

The stuck VALID is dropped only by RECOVERY.UNBLOCK, which means software has reset the peripheral and its AXI state is gone. STATUS exposes RESP_BUSY (the peripheral owes a response) and CMD_STUCK (it never accepted the command) so a driver can sequence this. Bound the busy poll: a peripheral that accepted and then died owes a response forever. A transaction arriving while blocked is rejected, not stalled, so drivers need a retry path.

Note that downstream_broken is independent of CTRL.AUTO_ISOLATE_EN. That bit governs only the visible ISOLATED status; blocking after a timeout is required for protocol safety and happens either way.

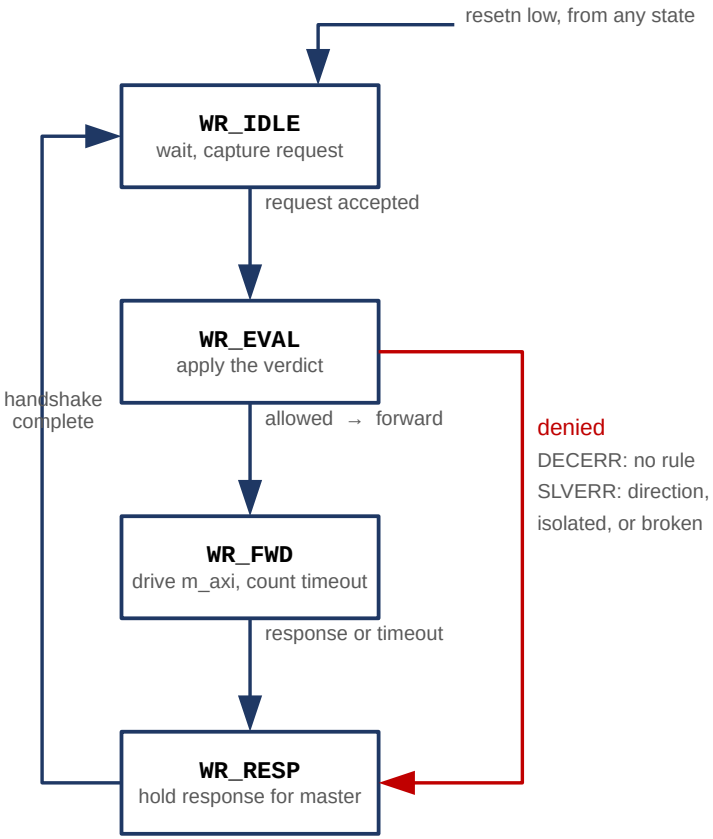
Key internal signals

Signal	Direction	Meaning
chk_w_addr / chk_r_addr	to regs	address presented to the rule lookup, one port per datapath
chk_*_match	from regs	some valid rule contains this address; low means DECERR
chk_*_allow	from regs	that rule permits this direction; low means SLVERR
global_enable	from regs	CTRL bit 0. Low is bypass: forward everything unchecked
isolate_effective	from regs	MANUAL_ISOLATE or the auto-isolate latch
forward_blocked	internal	isolate_effective or downstream_broken - blocks new forwarding
wr/rd_resp_busy	to regs	peripheral accepted the command and still owes a response
fault_* pulses	to regs	single-cycle, per direction; set sticky STATUS and capture FAULT_ADDR
unblock	from regs	pulse on RECOVERY.UNBLOCK; releases the block and discards a stuck VALID

3. Datapath state machines

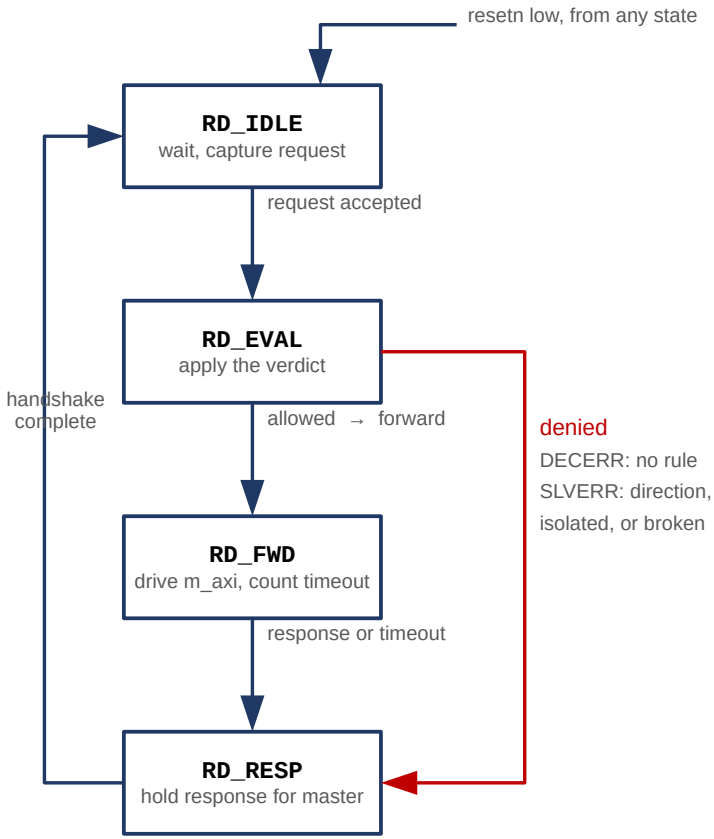
Both FSMs are enum-typed, so Questa names the states in its coverage report. All 14 transitions are covered by the test suite.

Write datapath (wr_state)



A transaction arriving while the downstream is blocked is answered SLVERR from EVAL - v2.0 removed the reset pulse, and with it the bounded window in which arrivals used to be stalled instead. The timeout in FWD covers both the address phase and the response phase.

Read datapath (rd_state)



Structurally identical, but a separate FSM with its own capture registers and lookup port. A denied read drives RDATA to zero before responding, so it cannot return stale data from an earlier permitted read.

On a timeout, m_axi_*VALID is deliberately NOT withdrawn.

AXI requires VALID to hold until READY. The FSM leaves FWD and answers the master, but the master-side VALID stays asserted until software writes RECOVERY.UNBLOCK - the one point where dropping it is legitimate, because software has just reset the peripheral and its protocol state no longer means anything.

3. Datapath state machines — description

State by state

IDLE waits for a request. The write path needs both AWVALID and WVALID before asserting either READY - a slave may always add wait states, so this is compliant and keeps one simple pattern. Address, protection bits, data and strobes are captured on entry to EVAL.

EVAL always one cycle. The rule lookup result for the captured address is already available combinationally, so the verdict costs no extra cycle. If the downstream is blocked the answer is SLVERR - v2.0 removed the reset pulse and with it the window in which arrivals were stalled instead.

FWD drives the master side and runs the timeout counter. The counter starts when forwarding starts and covers the whole round trip, so a peripheral that never raises AWREADY is caught by the same mechanism as one that accepts and then goes silent.

RESP asserts the response and holds it, with the payload stable, until the master's READY arrives.

Response encoding

Condition	Response	Status bit
allowed, peripheral answers	as returned	-
no valid rule matches	DECERR	ADDR_VIOLATION
matched, direction denied	SLVERR	PERM_VIOLATION
blocked while isolated	SLVERR	none
peripheral timed out	SLVERR	TIMEOUT_ERROR

A transaction blocked because the core is isolated returns SLVERR but sets no status bit and raises no interrupt. Only genuine violations and timeouts are logged, so a burst of rejected traffic during isolation cannot bury the fault that caused it.

Performance

Measured against a zero-wait-state peripheral, counting clock edges from request assertion to response valid:

```
single write   6 cycles
single read    6 cycles
```

The suite fails if either exceeds 8, so the figure cannot silently rot. Cost is per transaction and does not amortise.

FSM transition coverage (Questa 2024.1)

Transition	Hits	Exercised by
IDLE → EVAL	19 / 19	every transaction
EVAL → FWD	13 / 13	every permitted transaction
EVAL → RESP	5 / 5	denial tests C, E, J, K, L, M
EVAL → IDLE	1 / 1	test S - reset asserted during EVAL
FWD → RESP	10 / 10	normal completion and timeout tests F, N, Q, R
FWD → IDLE	3 / 3	test S - reset asserted during FWD
RESP → IDLE	15 / 15	every completed response

4. Register map and rule table

All registers are 32 bit and word aligned on s_axi_ctrl. Reset values are shown; the core comes up secure by default.

Fixed registers

Offset	Name	Access	Reset
0x00	CTRL	R/W	0x3 (enabled, auto-isolate on)
0x04	STATUS	R / W1C	0x0
0x08	IRQ_ENABLE	R/W	0x7 (all enabled)
0x0C	TIMEOUT_VALUE	R/W	all ones (effectively disabled)
0x10	FAULT_ADDR	R	0x0
0x14	FAULT_INFO	R	0x0
0x18	CORE_INFO	R	version (0x0200) and NUM_RULES
0x1C	RECOVERY	W	bit0 UNBLOCK, self-clearing

Rule table

Offset	Name	Description
0x40 + i·0x10	RULE_BASE[i]	inclusive base address
0x44 + i·0x10	RULE_LIMIT[i]	inclusive top address
0x48 + i·0x10	RULE_PERM[i]	permissions and valid bit

i runs 0 .. NUM_RULES-1. With the default 8 rules the table spans 0x40 - 0xBF. CTRL_ADDR_WIDTH must be wide enough to reach the whole table; a validation callback in hw.tcl enforces this, because an undersized control port silently makes high-index rules unreachable.

CTRL (0x00)

31:3	2	1	0
reserved	MANUAL_ISOLATE	AUTO_ISOLATE_EN	GLOBAL_ENABLE

STATUS (0x04) - bits 2:0 sticky, write 1 to clear

8:7	6:5	4	3	2:0
RD/WR_CMD_STUCK	RD/WR_RESP_BUSY	BLOCKED	ISOLATED	the three sticky bits

FAULT_INFO (0x14)

31:4	3:1	0
reserved	type: 1=ADDR 2=PERM 3=TIMEOUT	WAS_WRITE

RULE_PERM[i] (0x48 + i·0x10)

31:3	2	1	0
reserved	VALID	WRITE_ALLOW	READ_ALLOW

CORE_INFO (0x18)

31:16	15:8	7:0
version 0x0102 = v1.2	reserved	NUM_RULES

Clearing TIMEOUT_ERROR is not enough on its own

It also releases the auto-isolate latch. What it does NOT do, as of v2.0, is reopen the downstream - that takes an explicit RECOVERY.UNBLOCK after software has reset the peripheral. A v1.x driver stops here and then sees every access return SLVERR.

Reconfiguring a live rule

A rule is three registers, so updating one that is currently VALID leaves a window where BASE is new but LIMIT is still old. Clear VALID first (unnecessary at init - VALID is 0 out of reset):

```
write RULE_PERM[i] = 0
write RULE_BASE[i], RULE_LIMIT[i]
write RULE_PERM[i] = perms | VALID
```

4. Register map — description and software notes

Programming model

Bring-up is: program the rules, set a timeout, enable interrupts. The core is secure by default - GLOBAL_ENABLE resets to 1 and the rule table resets empty, so with no configuration at all every transaction is denied rather than passed.

TIMEOUT_VALUE resets to all ones, which is effectively no timeout. Set it to a real round-trip budget for your clock and peripheral, or fault isolation will not trigger in any useful time.

Interrupt handling

irq is a level interrupt, asserted while any enabled sticky STATUS bit is set. Clear at the source by writing 1 to the relevant bit; there is no separate acknowledge register. This is the standard memory-mapped-peripheral idiom and works directly with the Nios II HAL ISR pattern.

FAULT_ADDR and FAULT_INFO capture the most recent fault of any type. If a read and a write fault in the same cycle, both sticky bits set correctly but the capture registers take the write side.

Control port behaviour

s_axi_ctrl is single-outstanding and backpressures correctly: AWREADY/WREADY are withheld while a BVALID is unacknowledged, and ARREADY while an RVALID is. A driver that issues one access at a time sees no difference; one that pipelines simply waits.

Before v1.2 the port asserted READY on VALID arriving without checking whether the previous response had been accepted. A pipelined second access got its handshake taken, was silently dropped, and never received a response - a lost register write and a wedged channel. If you are carrying an older copy of this core, that is the one fix worth taking.

Writes to reserved or unaligned offsets are ignored and answered OKAY; reads of them return zero.

Parameters

Parameter	Default	Notes
ADDR_WIDTH	32	data-path address width
DATA_WIDTH	32	data-path data width; 32 or 64. Only 32 has been simulated
CTRL_ADDR_WIDTH	12	must cover 0x40 + NUM_RULES·16 bytes; checked by hw.tcl
NUM_RULES	8	rule table depth. Scales the combinational lookup - the likely critical path
TIMEOUT_WIDTH	20	max programmable timeout is 2^TIMEOUT_WIDTH - 1 clock cycles

The suite runs at the default parameter set only. Other configurations are checked by lint across the parameter space, which proves they elaborate, not that they work - the rule-table decode is exactly where an off-by-one would hide.